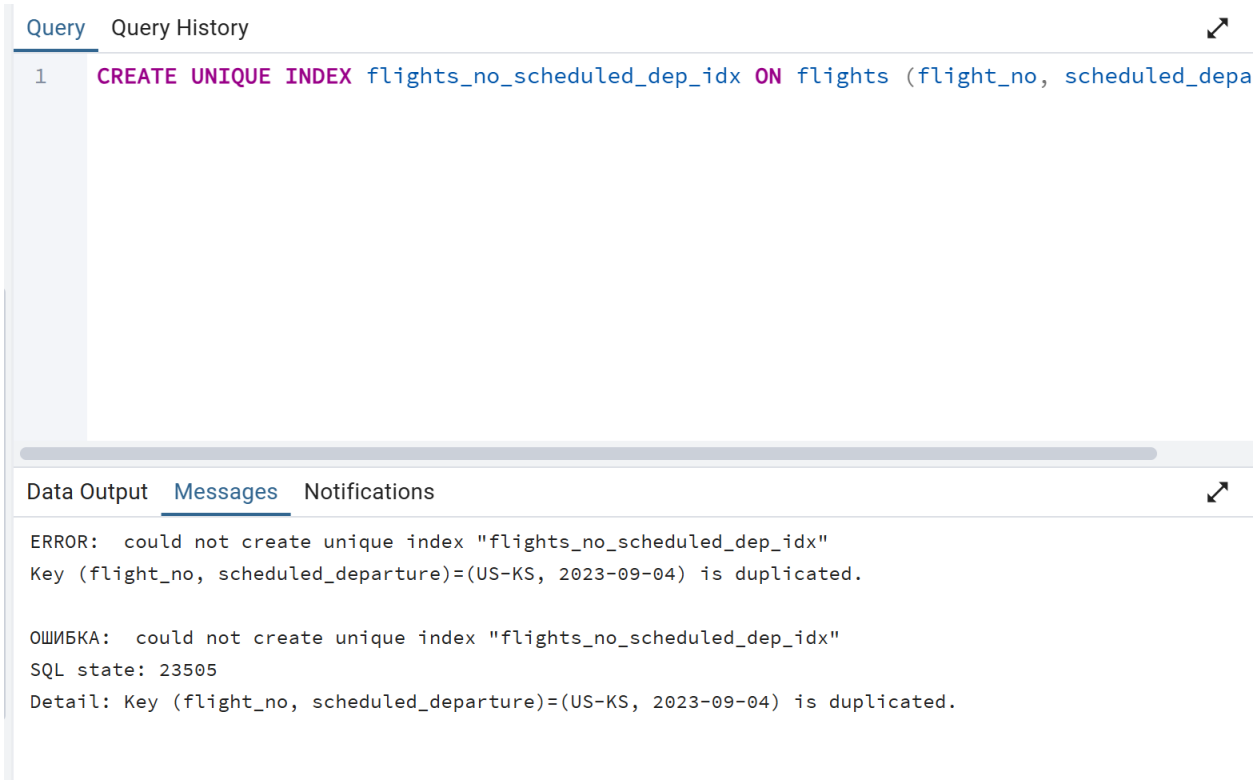
Laboratory work 7.

Task1. Create an index on the actual_departure column in the flights table.



Task2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.



Task3 Create a composite index on the departure_airport_id and arrival_airport_id columns.

Query Query History

```
1  CREATE INDEX flights_airports_idx ON flights (departure_airport_id, arrival_airport_id
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 157 msec.
```

Task4. Evaluate the difference in query performance with and without indexes. Measure performance differences.

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM flights
3  WHERE actual_departure = '2026-06-01 12:00:00';
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	QUERY PLAN text
3	Rows Removed by Filter: 1000
4	Buffers: shared hit=13
5	Planning:
6	Buffers: shared hit=6
7	Planning Time: 2.043 ms
8	Execution Time: 0.240 ms

Total rows: 8 Query complete 00:00:00.114 CRLF Ln 3, Col 48

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM flights
3  WHERE actual_departure = '2026-06-01 12:00:00';
```

Data Output | Messages | Notifications

Showing rows: 1 to 11 | Page No: 1 of 1

	QUERY PLAN text
7	Buffers: shared read=2
8	Planning:
9	Buffers: shared hit=16 read=1
10	Planning Time: 3.544 ms
11	Execution Time: 0.157 ms

Total rows: 11 | Query complete 00:00:00.101 | CRLF | Ln 3, Col 48

Task 5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM flights
3  WHERE departure_airport_id = 4
4    AND arrival_airport_id = 12;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	QUERY PLAN text
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=3
11	Planning Time: 0.385 ms
12	Execution Time: 0.211 ms

Total rows: 12 | Query complete 00:00:00.131 | CRLF | Ln 4, Col 30

Task 6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers. Explain in your own words what is going on in the output?

Query | Query History

```
1  CREATE UNIQUE INDEX passengers_passport_uniq_idx
2  ON passengers (passport_number);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 110 msec.
```

Query | Query History

```
1  SELECT indexname, indexdef
2  FROM pg_indexes
3  WHERE schemaname = 'public'
4    AND tablename = 'passengers';
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	indexname name	indexdef text
1	passengers_pkey	CREATE UNIQUE INDEX passengers_pkey ON public.passengers USING btree (passenger_id)
2	passengers_passport_uniq_i...	CREATE UNIQUE INDEX passengers_passport_uniq_idx ON public.passengers USING btree (passpo

Query Query History

```
1 INSERT INTO passengers (passenger_id,first_name, last_name,date_of_birth,gender,country_of_citizenship,country_of_residence,pas
2 VALUES (201, 'Alikhan', 'Tursynbek', '1987-03-27', 'Male', 'Czech Republic', 'Slovenia', '53296878-0', '2023-04-28', '2023-07-1
3 INSERT INTO passengers (passenger_id,first_name, last_name,date_of_birth,gender,country_of_citizenship,country_of_residence,pas
4 VALUES (202, 'Aizere', 'Tursynbek', '1987-03-27', 'Female', 'Czech Republic', 'Slovenia', '53296878-0', '2023-04-28', '2023-07-
```

Data Output Messages Notifications

```
ERROR:  повторяющееся значение ключа нарушает ограничение уникальности "passengers_passport_uniq_idx"
Ключ "(passport_number)=(53296878-0)" уже существует.

ОШИБКА:  повторяющееся значение ключа нарушает ограничение уникальности "passengers_passport_uniq_idx"
SQL state: 23505
Detail: Ключ "(passport_number)=(53296878-0)" уже существует.
```

The compiler will output error because of the duplicate passport numbers.

Task 7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give the explanation of the results.

Query Query History

```
1 CREATE INDEX passengers_compound_idx
2 ON passengers (first_name, last_name, date_of_birth, country_of_citizenship);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 159 msec.
```

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM passengers
3  WHERE country_of_citizenship = 'Philippines'
4    AND date_of_birth BETWEEN '1984-01-01' AND '1984-12-31';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on passengers (cost=0.00..6.50 rows=1 width=64) (actual time=0.035..0.065 rows=1.00 loops=1)
2	Filter: ((date_of_birth >= '1984-01-01'::date) AND (date_of_birth <= '1984-12-31'::date) AND ((country_of_citizenship)::text = 'Philip
3	Rows Removed by Filter: 199
4	Buffers: shared hit=3
5	Planning:
6	Buffers: shared hit=66 read=1

Total rows: 8 | Query complete 00:00:00.113 | CRLF | Ln 4, Col 59

*Because of the **leftmost prefix rule**, a composite index (first_name, last_name, date_of_birth, country_of_citizenship) can only be used if you search starting from the **first** column (left side).*

Since our query filtered by country_of_citizenship and date_of_birth (columns at the very end) while skipping the first columns, the database could not use the index and chose a full table scan (Seq Scan) instead.

Task8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.

Query | Query History

```sql
SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
  AND tablename = 'passengers';
 DROP INDEX passengers_passport_uniq_idx;
DROP INDEX passengers_compound_idx;
```

Data Output | Messages | Notifications

```
DROP INDEX

Query returned successfully in 131 msec.
```

Task9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.

Query | Query History

```sql
EXPLAIN ANALYZE
SELECT * FROM booking
WHERE passenger_id = 100;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page

	QUERY PLAN text
1	Seq Scan on booking (cost=0.00..17.25 rows=3 width=51) (actual time=0.111..0.192 rows=3.00 loop...
2	Filter: (passenger_id = 100)
3	Rows Removed by Filter: 498
4	Buffers: shared hit=11
5	Planning:
6	Buffers: shared hit=40
7	Planning Time: 31.419 ms
8	Execution Time: 0.222 ms

Query Query History

```
1  CREATE INDEX booking_passenger_id_idx
2  ON booking (passenger_id);
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 113 msec.
```

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM booking
3  WHERE passenger_id = 100;
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	QUERY PLAN text
4	Buffers: shared hit=2 read=1
5	-> Bitmap Index Scan on booking_passenger_id_idx (cost=0.00..4.17 rows=3 width=0) (actual time=0.074..0.074 rows=3.00 loo...
6	Index Cond: (passenger_id = 100)
7	Index Searches: 1
8	Buffers: shared read=1
9	Planning:
10	Buffers: shared hit=16 read=1
11	Planning Time: 2.531 ms
12	Execution Time: 0.126 ms

Total rows: 12 Query complete 00:00:00.120

1. Execution Plan Changes: *Before indexing, the query used a* ***Sequential Scan (Seq Scan)****; after indexing on passenger_id, it switched to an* ***Index Scan****, directly locating physical row addresses.*

2. Performance Impact:

Execution time dropped significantly because the database bypassed a full table scan.

Task10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.

Query | Query History

```
1  CREATE INDEX booking_flight_composite_idx
2  ON booking_flight (booking_id, flight_id);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 127 msec.
```

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT f.flight_no, b.price
3  FROM booking b
4  JOIN booking_flight bf ON b.booking_id = bf.booking_id
5  JOIN flights f ON bf.flight_id = f.flight_id
6  WHERE b.price > 9000;
```

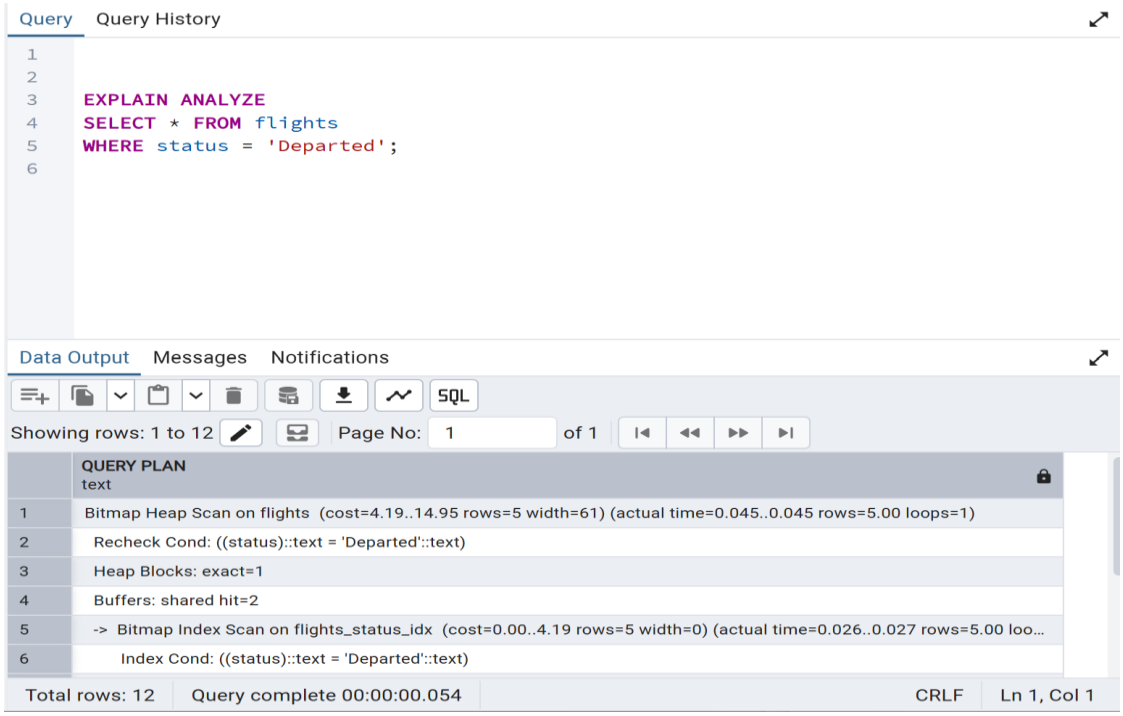
Data Output | Messages | Notifications

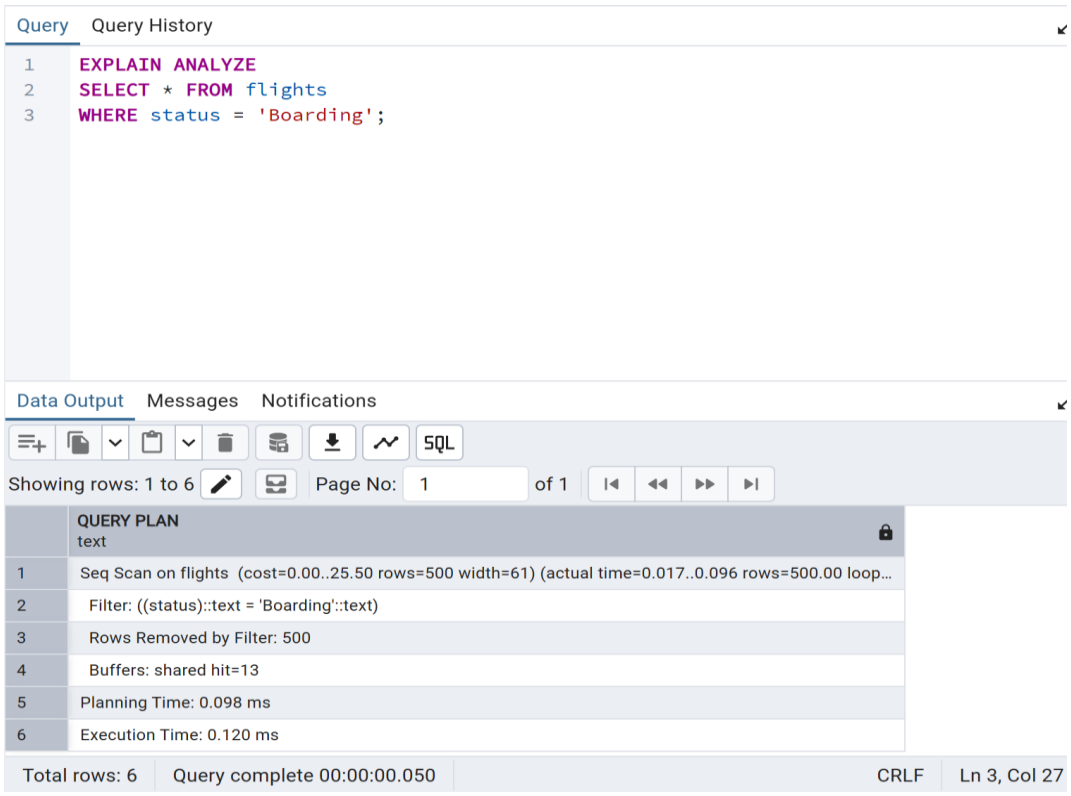
Showing rows: 1 to 24 | Page No: 1 of 1

	QUERY PLAN text
1	Hash Join (cost=39.16..67.11 rows=120 width=12) (actual time=0.185..0.276 rows=118.00 loops=1)
2	Hash Cond: (f.flight_id = bf.flight_id)
3	Buffers: shared hit=31
4	-> Seq Scan on flights f (cost=0.00..23.00 rows=1000 width=10) (actual time=0.012..0.049 rows=1000.00 loops=1)
5	Buffers: shared hit=13
6	-> Hash (cost=37.66..37.66 rows=120 width=10) (actual time=0.160..0.161 rows=118.00 loops=1)

Total rows: 24 | Query complete 00:00:00.048 | CRLF | Ln 6, Col 22

Task11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).





Why the optimizer ignores the index for some values: If a value appears in the majority of the rows (like 'Departed'), fetching them one by one through an index forces the database to constantly jump back and forth across the disk (random I/O). This is much slower than simply reading the entire table sequentially in one pass (Seq Scan), so the optimizer chooses to ignore the index. For rare values (like 'Boarding'), the index is used because it immediately points to just a tiny fraction of the rows.

Task12. Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index usage.

Query | Query History

```
1
2  CREATE INDEX baggage_weight_idx ON baggage (weight_in_kg);
3
4  EXPLAIN ANALYZE
5  SELECT * FROM baggage
6  WHERE weight_in_kg > 49;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	QUERY PLAN text
1	Bitmap Heap Scan on baggage (cost=4.31..8.37 rows=5 width=22) (actual time=0.018..0.020 rows=7.00 loops=1)
2	Recheck Cond: (weight_in_kg > '49'::numeric)
3	Heap Blocks: exact=4
4	Buffers: shared hit=6
5	-> Bitmap Index Scan on baggage_weight_idx (cost=0.00..4.31 rows=5 width=0) (actual time=0.005..0.005 rows=7.00 loo...
6	Index Cond: (weight_in_kg > '49'::numeric)

Total rows: 12 | Query complete 00:00:00.096 | CRLF | Ln 3, Col 1

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM baggage
3  WHERE weight_in_kg > 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on baggage (cost=0.00..10.25 rows=451 width=22) (actual time=0.011..0.044 rows=451.00 loop...
2	Filter: (weight_in_kg > '5'::numeric)
3	Rows Removed by Filter: 49
4	Buffers: shared hit=4
5	Planning:
6	Buffers: shared hit=17 dirtied=2

Total rows: 8 | Query complete 00:00:00.059 | CRLF | Ln 3, Col 24

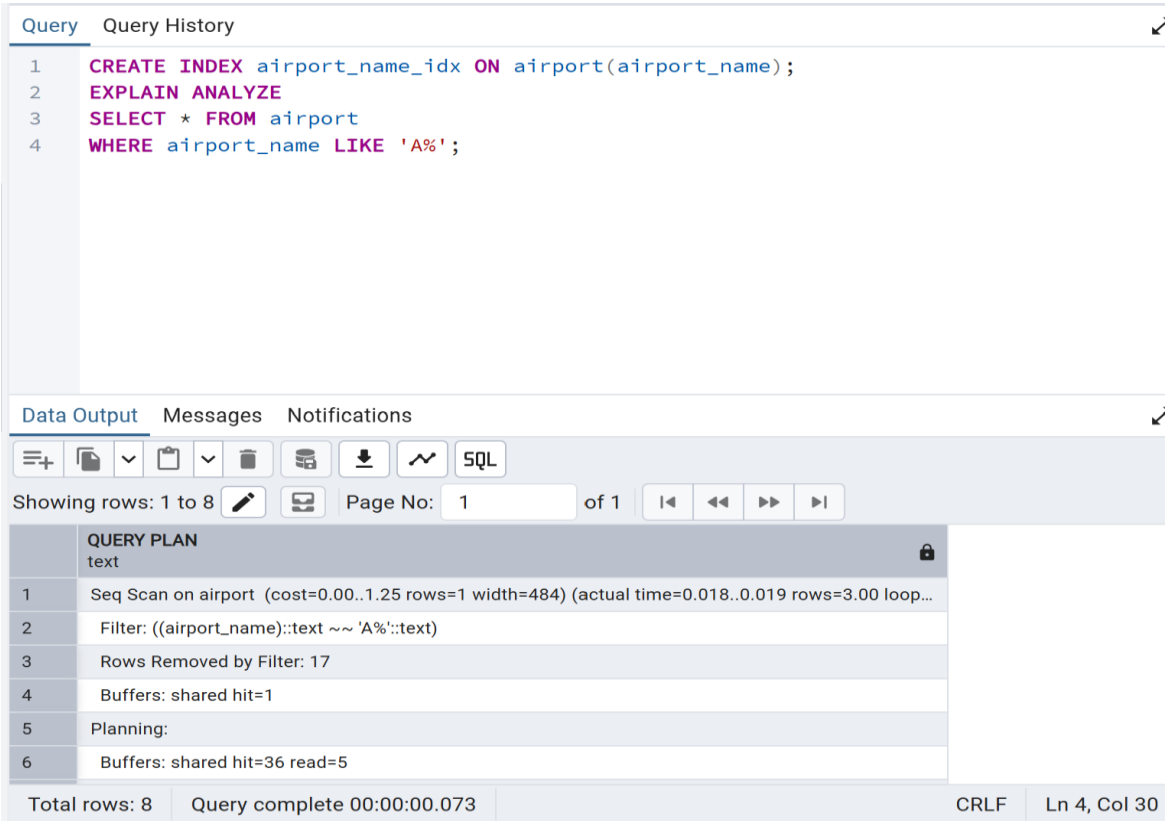
1. Query (weight_in_kg > 49 - Small Fraction):

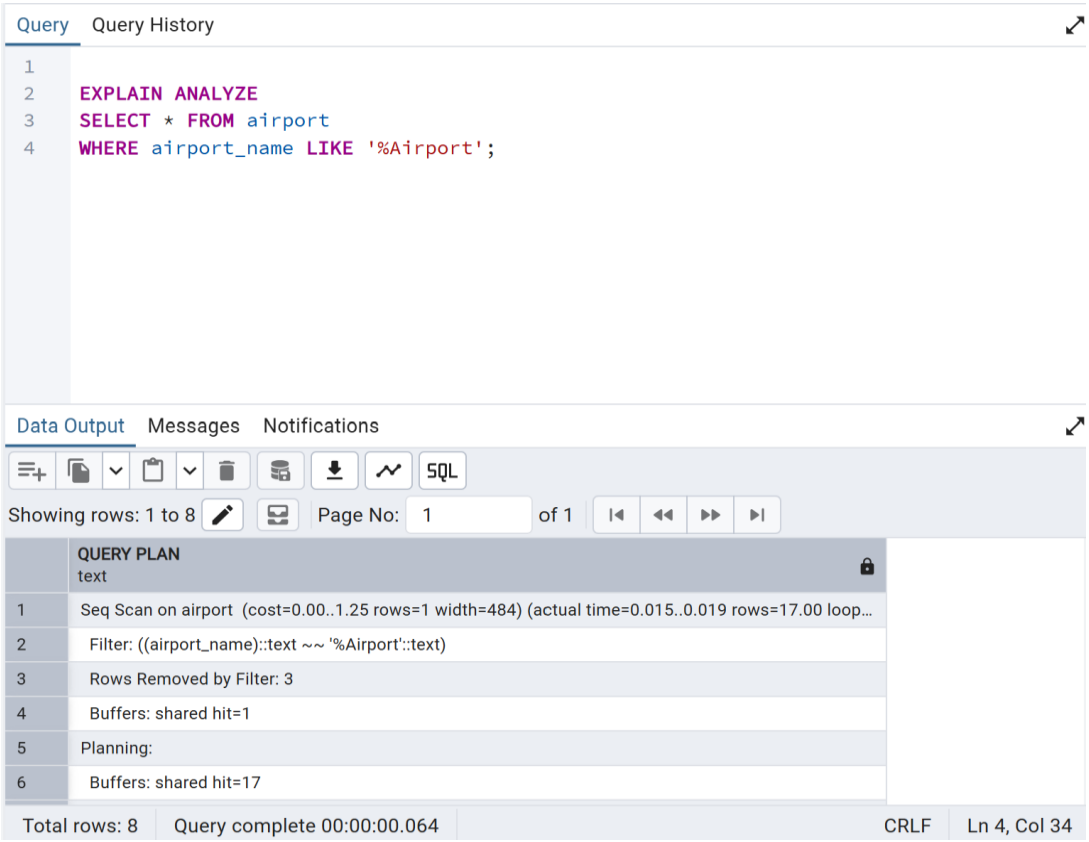
Because this condition matches only a small percentage of rows, the database optimizer chooses an ***Index Scan****. It is much faster to look up pointers in the index and jump straight to the few matching rows than to read the entire table.*

2. Query (weight_in_kg > 5 - Almost Whole Table):

Since this condition matches nearly all rows in the table, using the index would cause excessive random disk I/O (fetching index pointers followed by table pages). Therefore, the optimizer ignores the index and switches to a ***Sequential Scan (Seq Scan)****, reading the table sequentially, which is much more efficient for large datasets.*

Task13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are different.





LIKE 'A%' (Index Used): *Because the wildcard % is at the end, the search has a* ***prefix****. The database can use the B-tree index to instantly jump to the exact alphabetical range where airport names start with 'A'.*

LIKE '%Airport' (Index Ignored / Seq Scan): *Because the wildcard % is at the beginning, the database cannot know where the matching strings are located in the index tree. To check every airport name for a ending substring, it is forced to perform a full* ***Sequential Scan*** *(Seq Scan), ignoring the index.*

Task14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.

Query | Query History

```
1
2  CREATE INDEX airport_country_city_idx ON airport (country, city);
3
4  EXPLAIN ANALYZE
5  SELECT * FROM airport
6  WHERE country = 'Kazakhstan';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=484) (actual time=0.011..0.011 rows=0.00 loop...
2	Filter: ((country)::text = 'Kazakhstan'::text)
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=5 read=1

Total rows: 8 | Query complete 00:00:00.064 | CRLF | Ln 3, Col 1

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM airport
3  WHERE city = 'Almaty';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=484) (actual time=0.019..0.019 rows=0.00 loop...
2	Filter: ((city)::text = 'Almaty'::text)
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=20

Successfully run. Total query runtime: 81 msec. 8 rows affected.

Total rows: 8 | Query complete 00:00:00.081 | CRLF | Ln 3, Col 23

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM airport
3  WHERE country = 'Kazakhstan' AND city = 'Almaty';
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on airport (cost=0.00..1.30 rows=1 width=484) (actual time=0.022..0.022 rows=0.00 loop...
2	Filter: (((country)::text = 'Kazakhstan'::text) AND ((city)::text = 'Almaty'::text))
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning Time: 0.117 ms
6	Execution Time: 0.039 ms

Total rows: 6 Query complete 00:00:00.056 CRLF Ln 3, Col 50

Filtering by Country only (Seq Scan / Index Scan depending on size): *Because country is the first (leftmost) column in the composite index (country, city), the database* ***can*** *use the index. However, if the table is too small (like having only 20 rows) or if the database statistics aren't updated, the optimizer may choose a Seq Scan because it's faster than index overhead. If there were enough rows and proper data distribution, filtering by the leading column (country) would successfully execute via an Index Scan.*
Filtering by City only (Seq Scan): *The index is ignored because of the* ***leftmost prefix rule****—you cannot use a composite index if you skip the leading column (country). Therefore, the database is forced to perform a sequential scan.* ***Filtering by Both:*** *Utilizes the composite index effectively, filtering by country first and narrowing down by city.*

Task15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE LOWER(last_name) = 'irnis' and a query written as WHERE last_name = 'Irnis'. Explain which query can use the index and why.

Query | Query History

```
1  CREATE INDEX passengers_lower_lastname_idx ON passengers (LOWER(last_name));
2
3  EXPLAIN ANALYZE
4  SELECT * FROM passengers
5  WHERE LOWER(last_name) = 'irnis';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on passengers (cost=0.00..6.00 rows=1 width=64) (actual time=0.010..0.039 rows=1.00 loop...
2	Filter: (lower((last_name)::text) = 'irnis'::text)
3	Rows Removed by Filter: 199
4	Buffers: shared hit=3
5	Planning:
6	Buffers: shared hit=8 read=1

Successfully run. Total query runtime: 97 msec. 8 rows affected.

Total rows: 8 | Query complete 00:00:00.097 | CRLF | Ln 5, Col 34

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM passengers
3  WHERE last_name = 'Irnis';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on passengers (cost=0.00..5.50 rows=1 width=64) (actual time=0.011..0.022 rows=1.00 loop...
2	Filter: ((last_name)::text = 'Irnis'::text)
3	Rows Removed by Filter: 199
4	Buffers: shared hit=3
5	Planning:
6	Buffers: shared hit=17

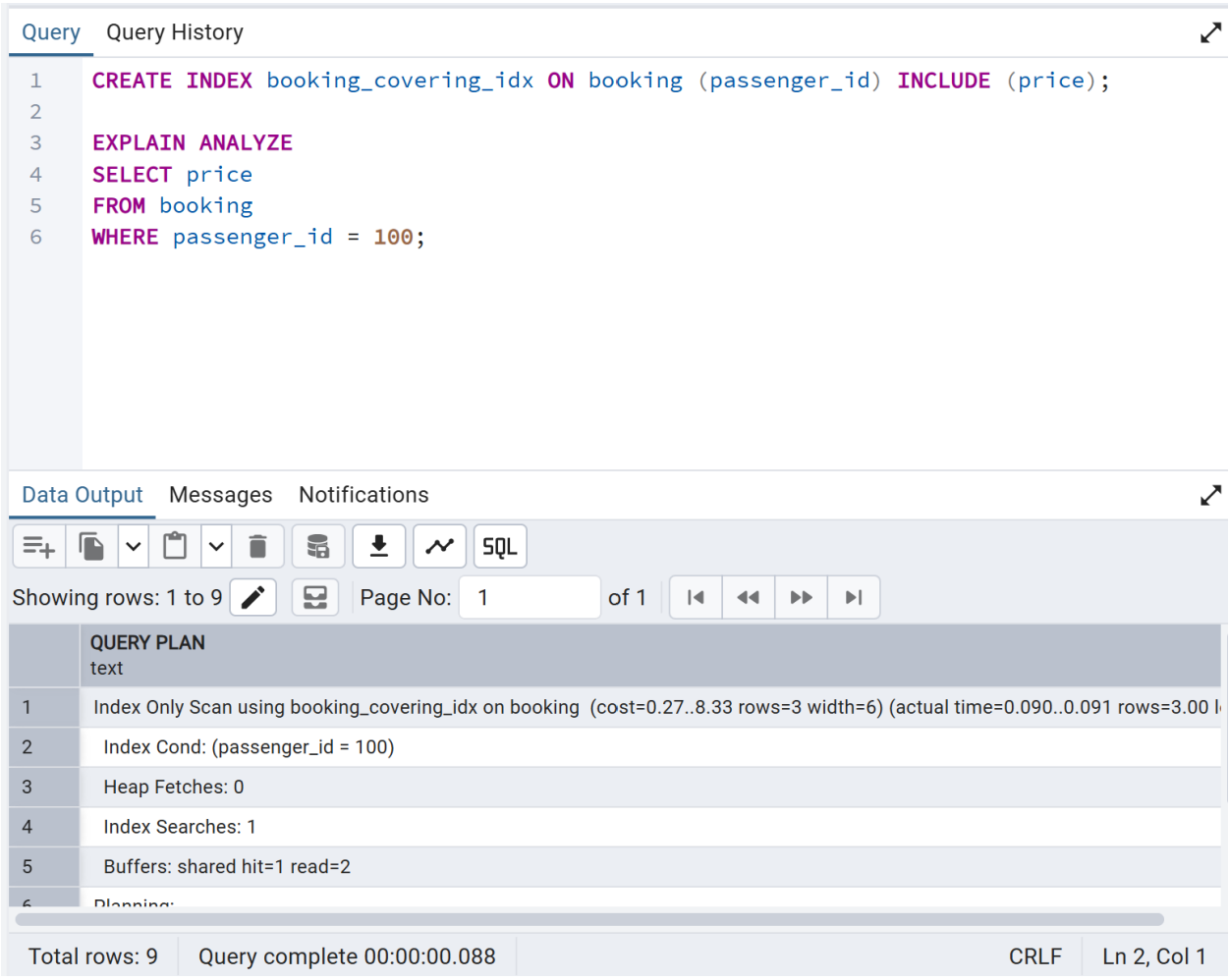
Total rows: 8 | Query complete 00:00:00.068 | CRLF | Ln 3, Col 27

WHERE LOWER(last_name) = 'irnis': *This query is designed to use the expression index because its condition precisely matches the indexed expression (LOWER(last_name)). However, if the table is very small (e.g., only 200 rows), the database optimizer bypasses the index and chooses a Seq Scan because sequential reading of a tiny dataset is faster than index overhead.*

WHERE last_name = 'Irnis': *This query forces a Seq Scan because it filters on the raw column instead of the expression, meaning the functional index cannot be applied.*

Why Task 9 used the index on 200 rows while Task 15 didn't?

Task16. Create a covering index on the booking table for the columns passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefit of a covering index.



Query Execution: *When running SELECT price FROM booking WHERE passenger_id = 100, the execution plan shows an* ***Index Only Scan*** *(or indicates Using index).*

Why it happens: *Because the index was created as CREATE INDEX ... ON booking (passenger_id) INCLUDE (price), the price column is stored directly inside the index leaf nodes. The database engine does not need to visit the actual table blocks (heap fetches) to retrieve the price.*

Benefit of a covering index: *It drastically reduces disk I/O and improves query performance by storing all requested columns inside the index structure, allowing the database to satisfy the query entirely from the index without touching the main table.*

Task17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.

Query | Query History

```
1
2   CREATE UNIQUE INDEX airline_code_uq_idx ON airline (airline_code);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 68 msec.
```